MRT3216

2026 Season — Advanced Diagnostics Report

This report covers in-depth diagnostic analyses drawn from AdvantageKit .wpilog match logs. Topics covered: command activity timelines, shooter spin-up timing, drive speed utilization, hood angle unit verification, loop cycle overruns, vision acceptance rates, battery discharge curves, and match-to-match learning trends.

Section	Topic	Key Question
2	Command Timeline Gantt	What was the robot actually doing each match?
3	Shooter Velocity Timing	Was the flywheel spun up before shooting?
4	Drive Speed Utilization	How aggressively was the robot driven?
5	Hood Angle Unit Investigation	Are hood setpoints in degrees or radians?
6	Loop Cycle Time	Were 20ms deadlines being met?
7	Vision Acceptance Rate	How often did pose estimates get accepted?
8	Battery Discharge Curve	How fast was charge depleted per match?
9	Match-to-Match Trends	Did performance change across quals?
Bonus	CAN Bus Health	Was the CAN bus a reliability risk?

Generated March 18, 2026 at 16:06 | MRT3216 Programming Subteam

Section 2 — Command Activity Timeline

Each bar represents a time interval where that command was actively scheduled during teleop. This is read directly from */RealOutputs/CommandsAll/** boolean signals. Note that 'StopHold' commands run as default commands — they occupy the subsystem when nothing else has it.

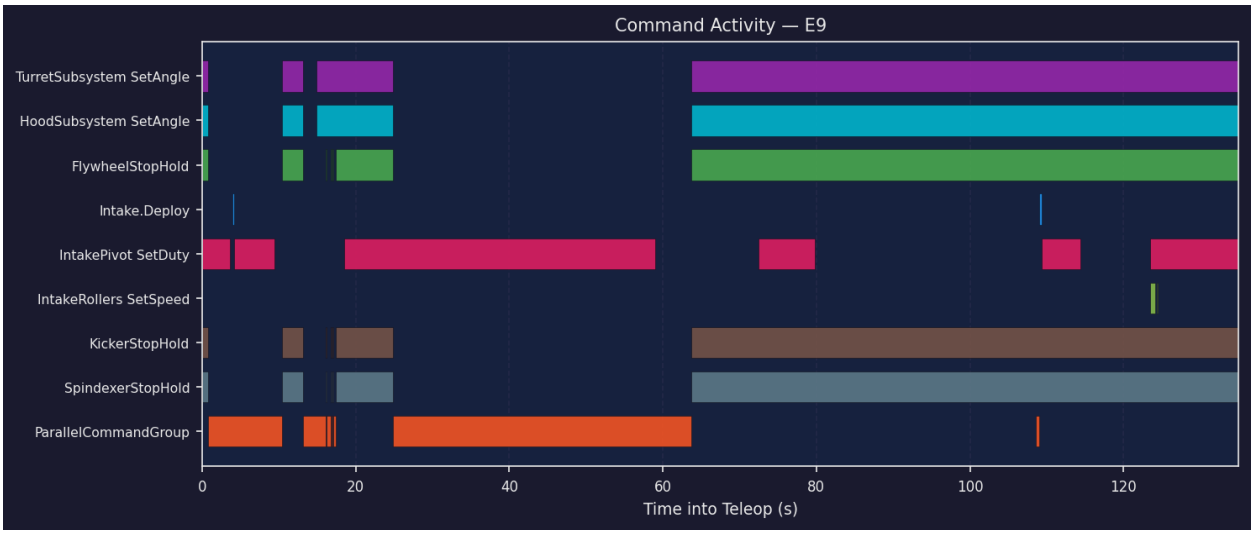

Figure — Command activity timeline for E9. X-axis is seconds into teleop.

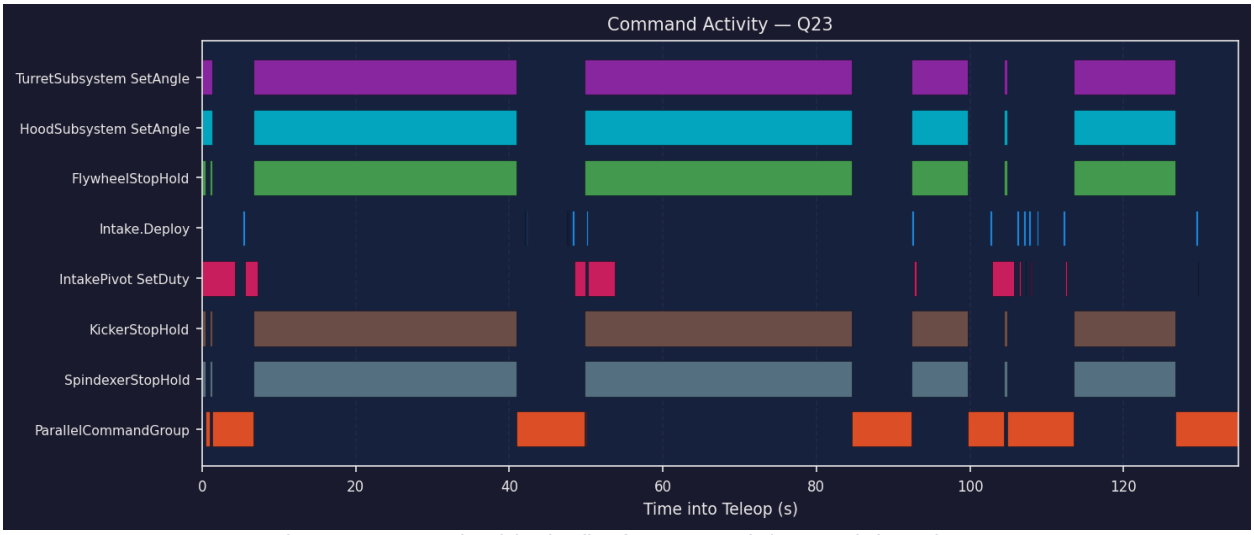

Figure — Command activity timeline for Q23. X-axis is seconds into teleop.

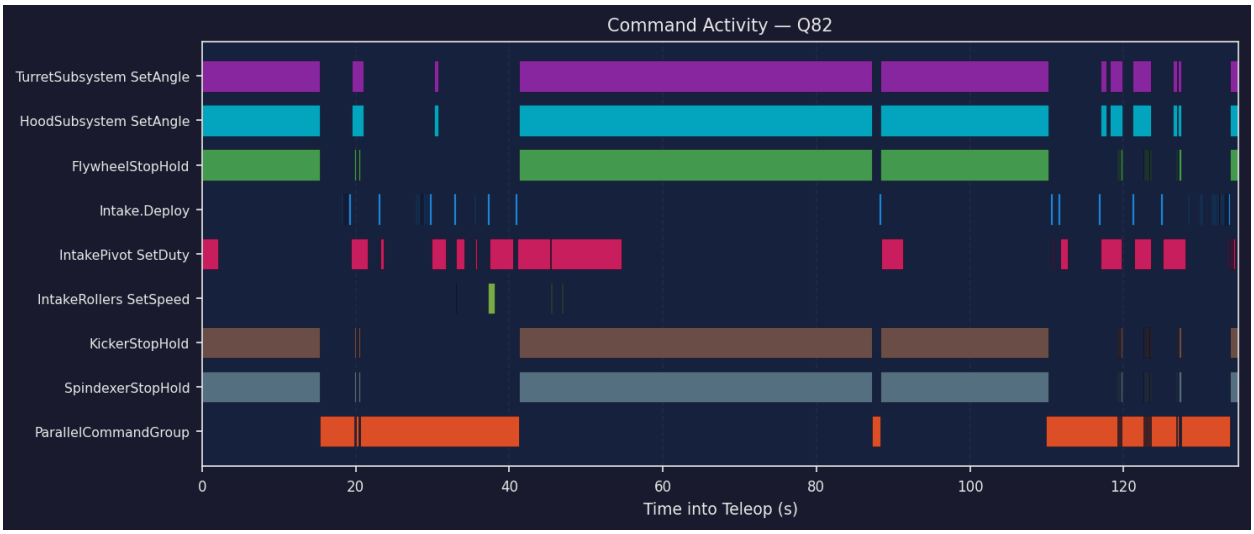

Figure — Command activity timeline for Q82. X-axis is seconds into teleop.

Command Usage Frequency (across all matches)

Command	Matches Present	Avg % Teleop Active
TurretSubsystem SetAngle	13	53.8%
HoodSubsystem SetAngle	15	53.3%
FlywheelStopHold	15	52.5%
InterruptShooting	15	0.0%
Left Bum Rush	0	0.0%
Intake.Deploy	11	50.2%
IntakePivot SetDuty	15	51.9%
IntakeRollers SetSpeed	5	50.0%
KickerStopHold	15	52.5%
SpindexerStopHold	15	52.5%
ParallelCommandGroup	15	51.8%

Section 3 — Shooter Velocity & Spin-up Timing

A critical cycle-time factor is whether the flywheel reached its target velocity before the kicker fired. If the kicker fires while the flywheel is still accelerating, the ball exits at reduced velocity and unpredictable angle. The charts below show flywheel velocity vs. setpoint for selected matches, plus a histogram of spin-up durations across all matches.

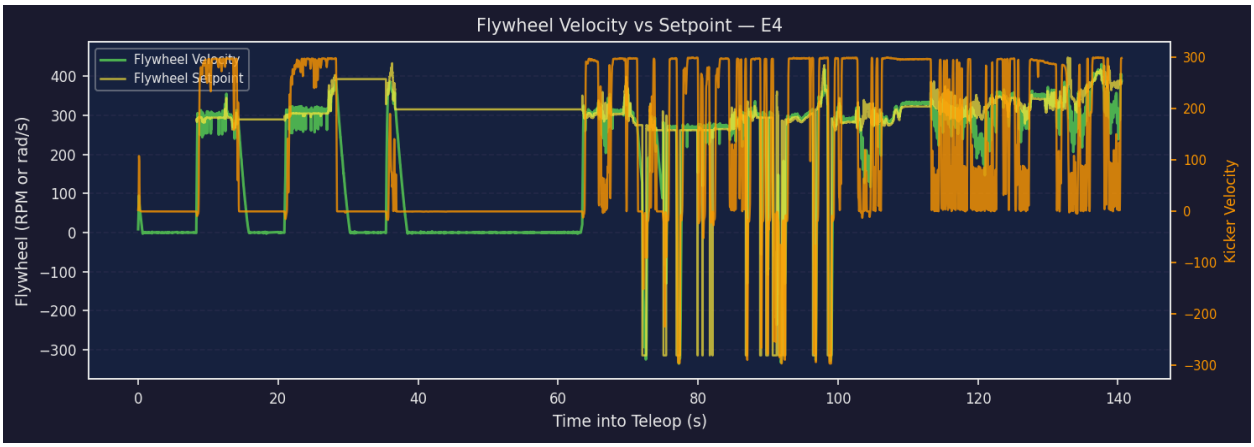

Figure — Flywheel velocity (green) vs setpoint (yellow) + kicker velocity (orange) for E4.

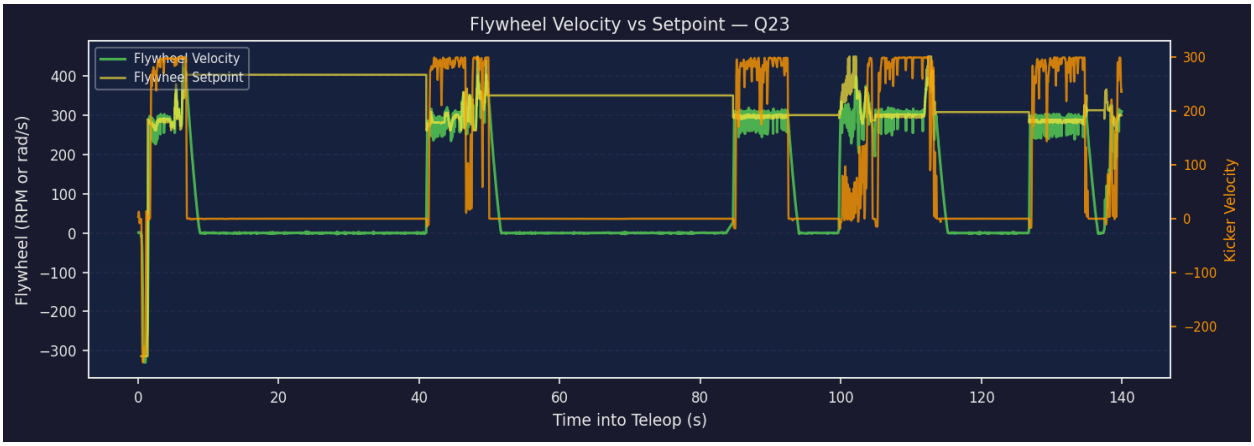

Figure — Flywheel velocity (green) vs setpoint (yellow) + kicker velocity (orange) for Q23.

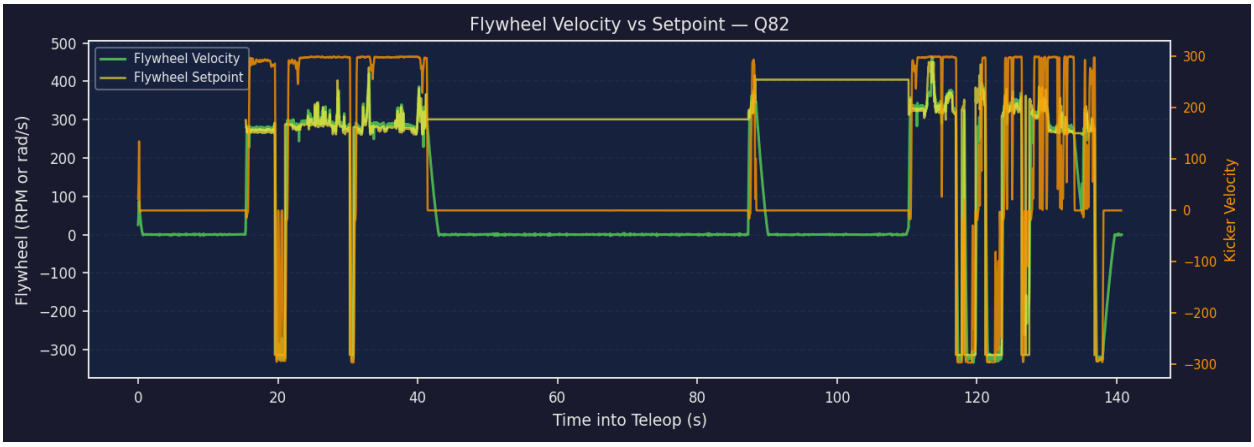

Figure — Flywheel velocity (green) vs setpoint (yellow) + kicker velocity (orange) for Q82.

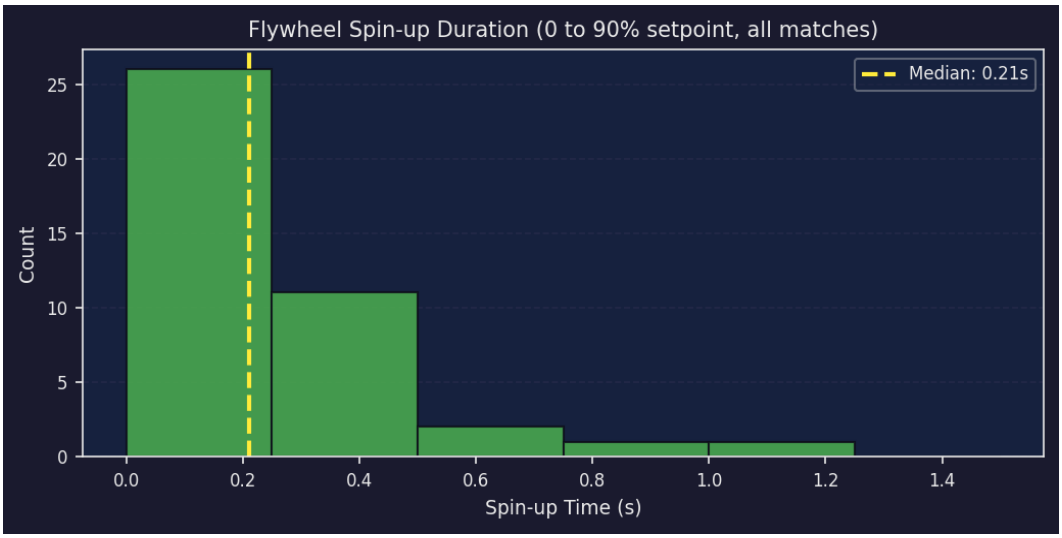

Figure — Distribution of flywheel spin-up times (0 to 90% setpoint) across all matches. Shorter is better.

Section 4 — Drive Speed Utilization

Understanding how aggressively the robot was driven helps distinguish 'high drive current from speed' vs. 'high drive current despite being slow'. Speed is computed from *SwerveChassisSpeeds/Measured* as sqrt($vx^2 + vy^2$) and normalized to the robot's theoretical maximum (5.05 m/s). The joystick axis value structure (*DriverStation/Joystick0/AxisValues*) is also available for future driver input analysis.

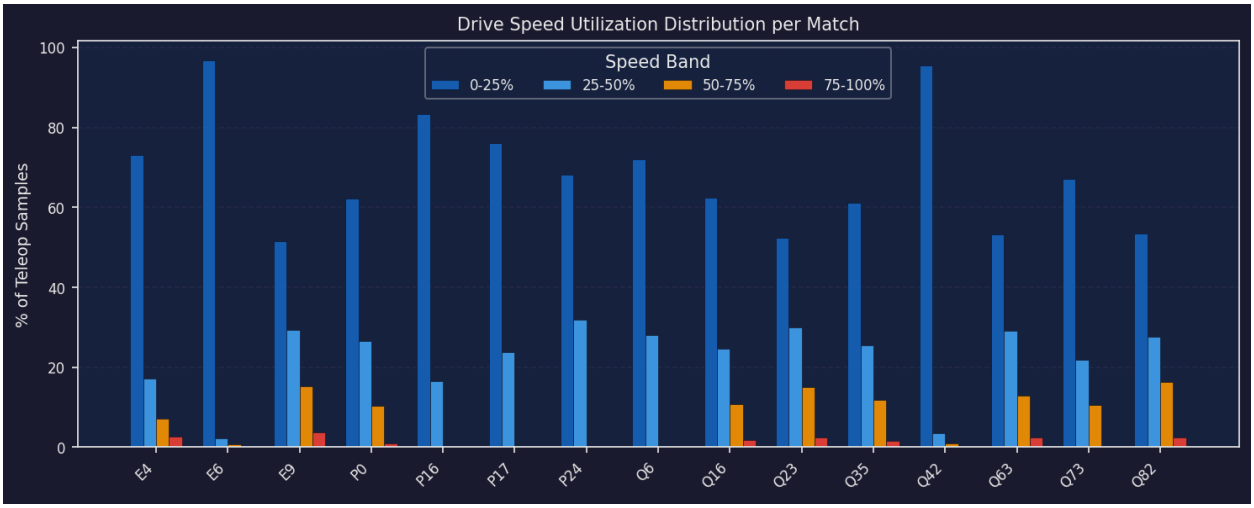

Figure — Speed utilization distribution. Each group of 4 bars shows what fraction of teleop the robot spent in each speed band.

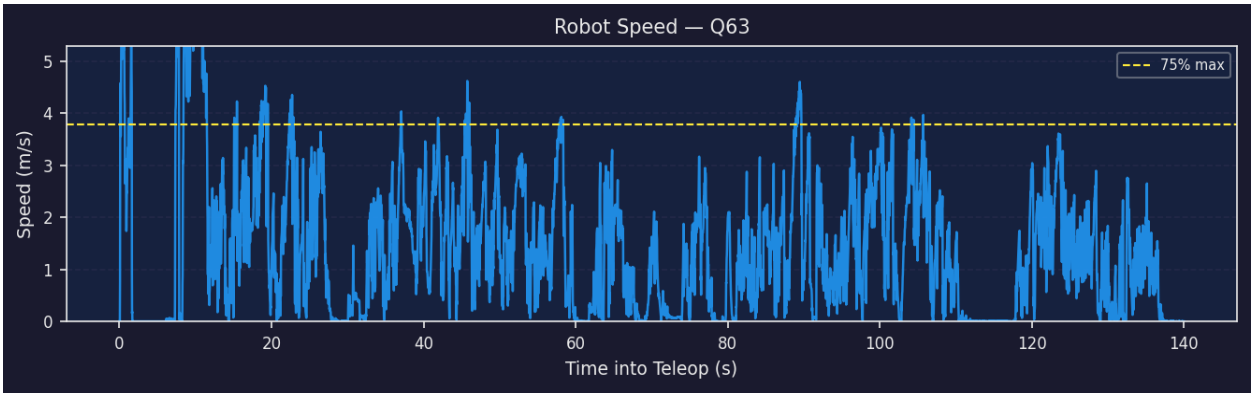

Figure — Robot speed over teleop for Q63.

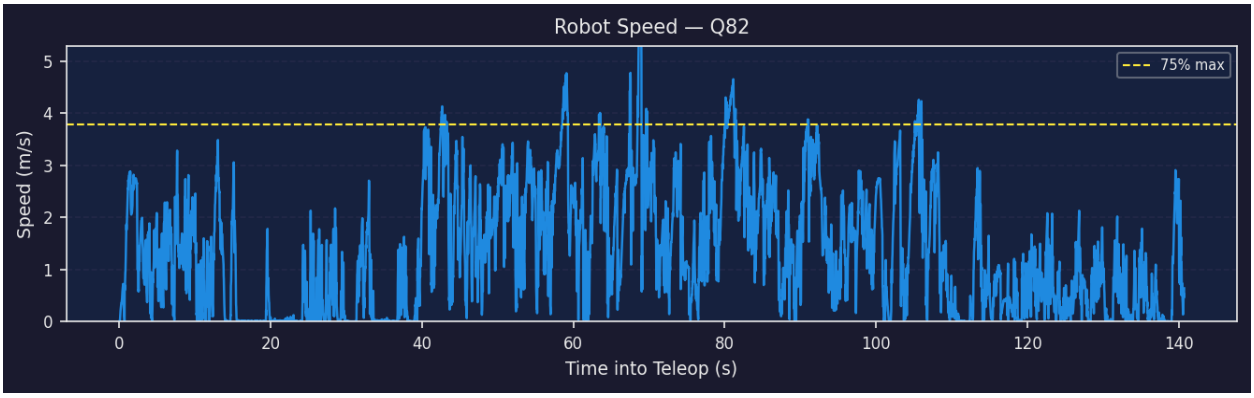

Figure — Robot speed over teleop for Q82.

Section 5 — Hood Angle Unit Investigation

Earlier single-match analysis found *Hood/Angle* values near 0.227 — which would be radians (~13°) rather than degrees. However, *RealOutputs/Hood/FX/PositionDegrees* logs the raw TalonFX position directly in degrees. Comparing both signals reveals whether the YAMS-layer signal is in radians or degrees.

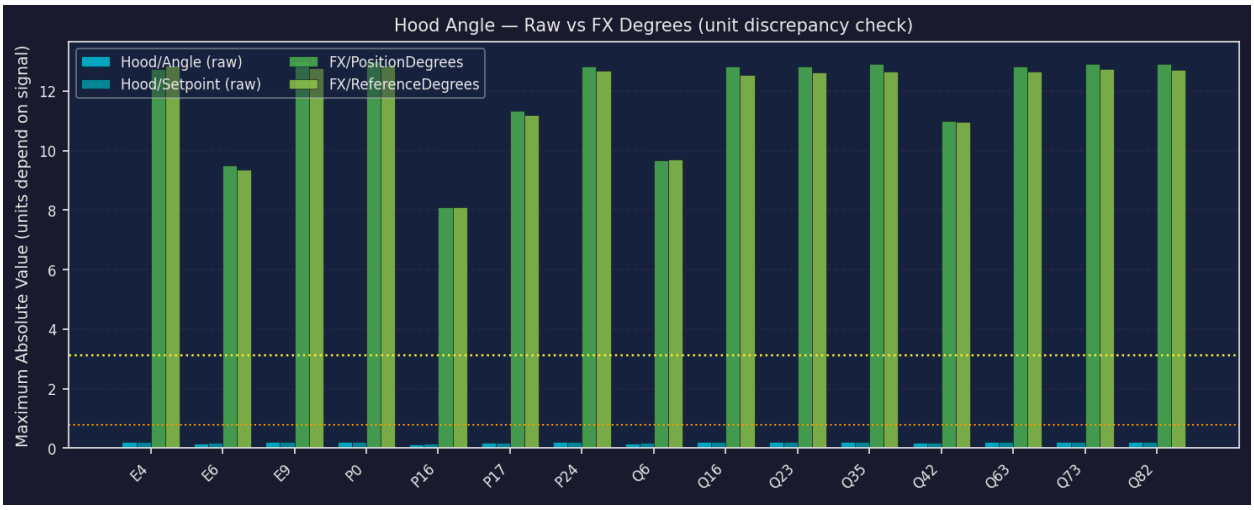

Figure — Max absolute value of all four Hood signals per match. If Hood/Angle peaks at ~0.2–0.5 while FX/PositionDegrees peaks at 10–30+, the YAMS signal is in radians.

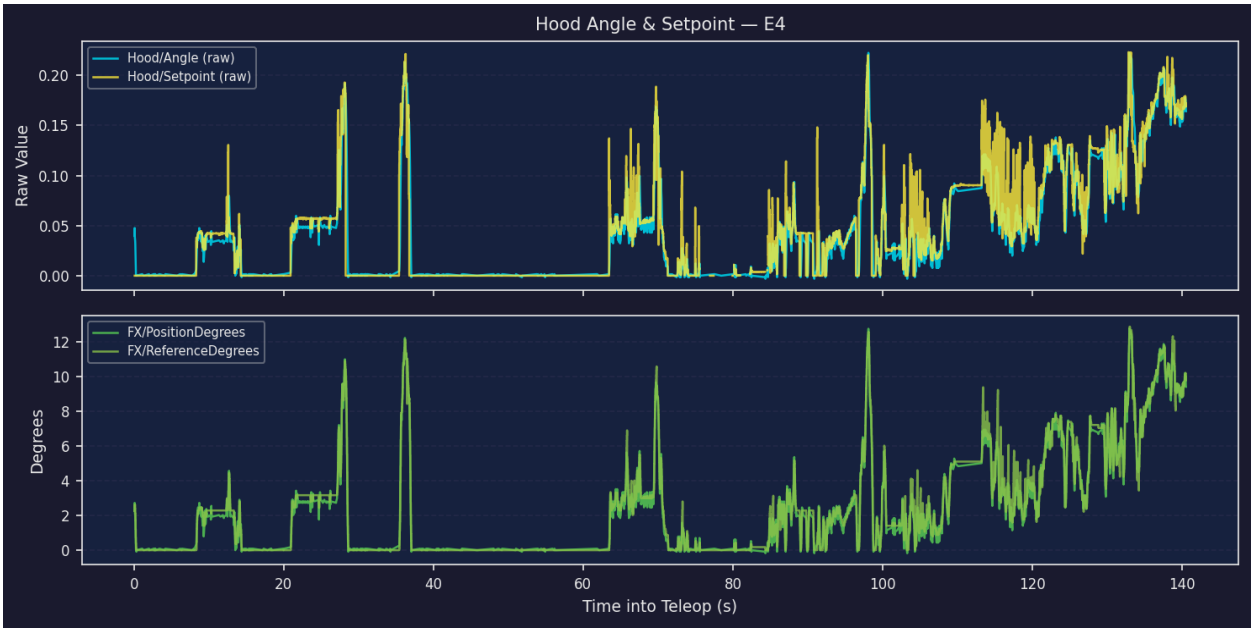

Figure — Hood angle and setpoint timelines for E4. Top panel: YAMS raw value. Bottom panel: TalonFX degrees.

Section 6 — Loop Cycle Time & Overruns

The roboRIO runs the robot loop on a 20ms (50Hz) period. If *FullCycleMS* exceeds 20ms, the scheduler misses its deadline, causing delayed sensor reads, stale odometry, and degraded control. *UserCodeMS* is the time spent executing user subsystem/command code. The difference between FullCycleMS and UserCodeMS is AdvantageKit logging overhead.

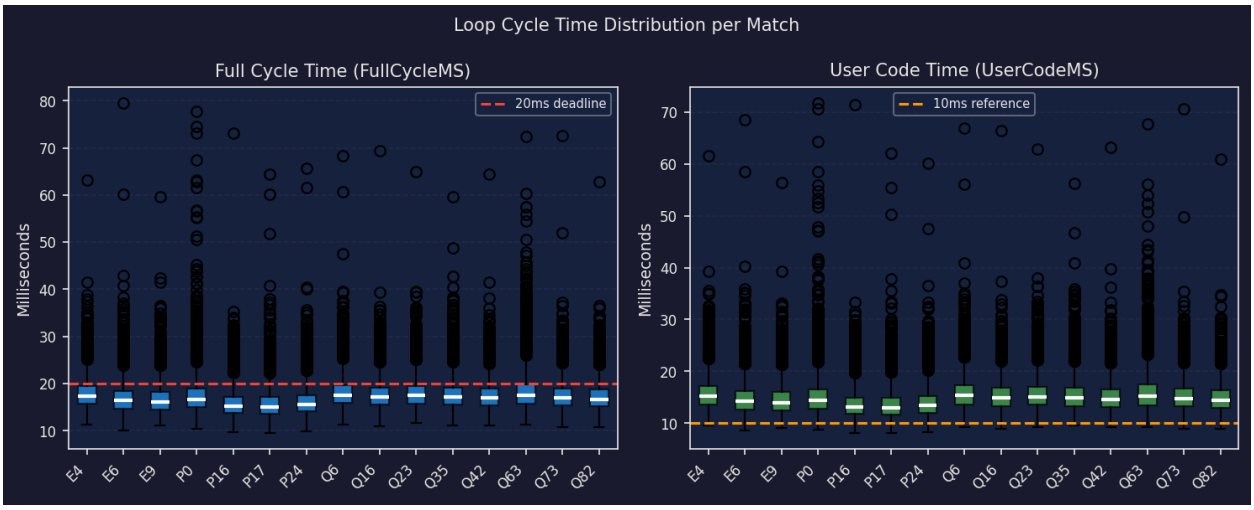

Figure — Box plots of FullCycleMS (left) and UserCodeMS (right). The red dashed line marks the 20ms deadline.

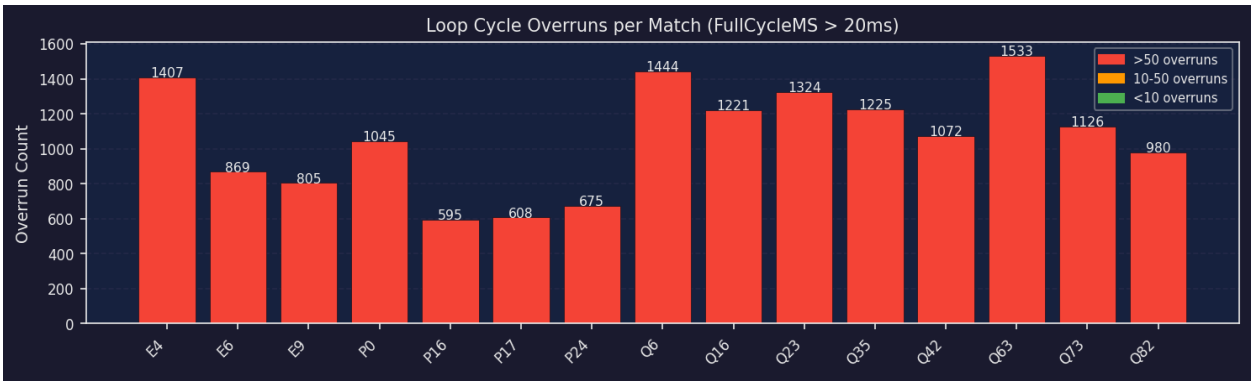

Figure — Count of loop cycles exceeding 20ms per match.

Match	Avg FullCycleMS	Max FullCycleMS	Avg UserCodeMS	Overruns (>20ms)
E4	17.90	63.20	15.66	1407
E6	16.87	79.47	14.74	869
E9	16.58	59.55	14.46	805
P0	17.36	77.82	15.10	1045
P16	15.85	73.07	13.71	595
P17	15.75	64.33	13.64	608
P24	16.16	65.72	13.98	675
Q6	18.02	68.36	15.72	1444
Q16	17.58	69.41	15.33	1221
Q23	17.86	64.89	15.56	1324
Q35	17.64	59.56	15.38	1225

Q42	17.32	64.48	15.07	1072
Q63	18.14	72.44	15.86	1533
Q73	17.48	72.57	15.23	1126
Q82	17.11	62.79	14.89	980

Section 7 — Vision Pose Acceptance Rate

Vision pose estimates are filtered by the Vision subsystem before being passed to the pose estimator. Factors that cause rejection include: excessive Z coordinate (camera disconnected or tag partially visible), pose outside field boundaries, and high tag ambiguity. A low acceptance rate means the robot is effectively driving without vision correction — relying entirely on wheel odometry, which drifts significantly over a 2-minute match.

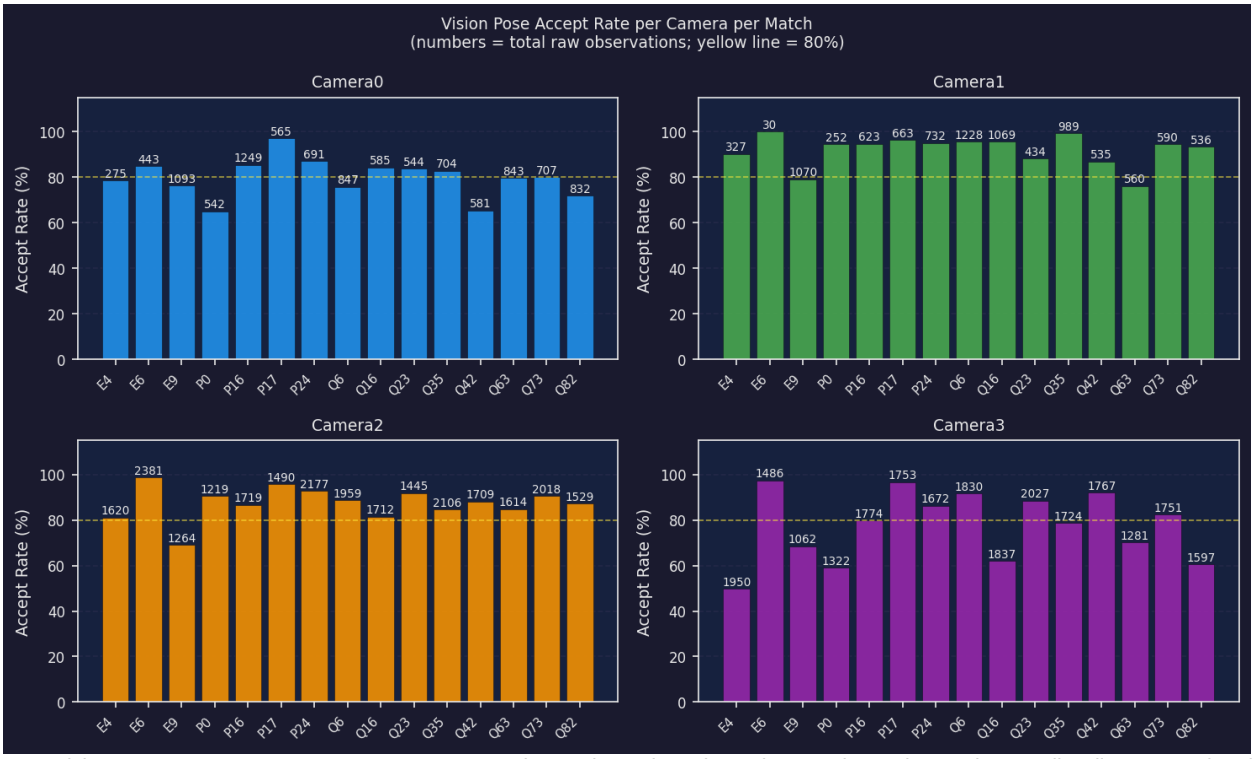

Figure — Vision pose acceptance rate per camera per match. Numbers above bars show total raw observations. Yellow line = 80% threshold.

Note: OrangePI-Left@2 and OrangePI-Right@1 are visible in SystemStats/NTClients, confirming two coprocessors are connected and publishing to NetworkTables. Camera0–3 correspond to the physical cameras on those coprocessors.

Section 8 — Battery Discharge Curves

By integrating *SystemStats/BatteryCurrent* over time (trapezoidal rule), we can construct a discharge curve: battery voltage as a function of cumulative Ah drawn. A healthy FRC battery (18 Ah rated, ~12–14 Ah usable) should show a relatively flat curve above 10V until the final few Ah. A steeply dropping curve indicates either a degraded battery or excessive current draw.

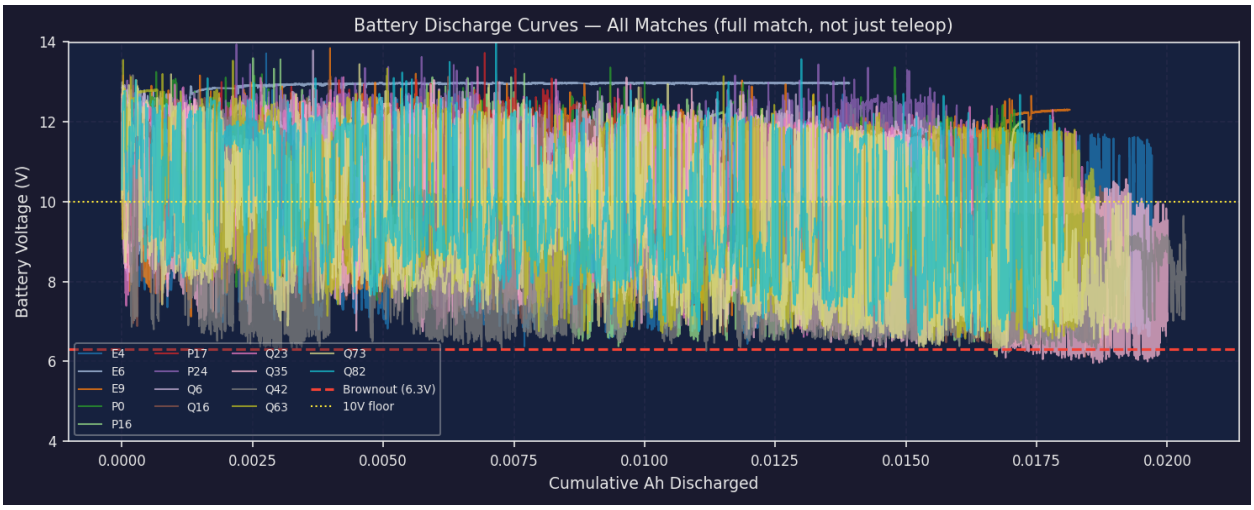

Figure — All matches overlaid. A curve that drops quickly to 6–8V with only 1–2 Ah drawn indicates a heavily loaded or partially discharged battery.

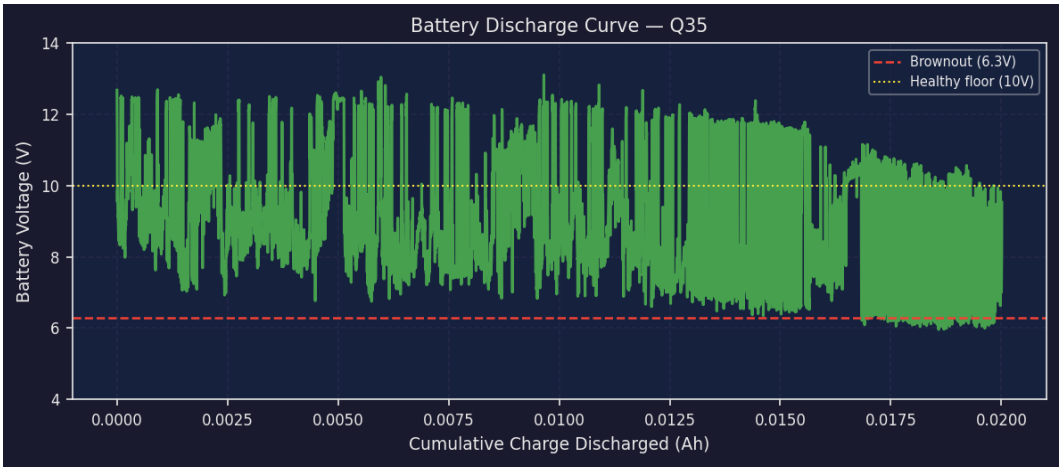

Figure — Discharge curve for Q35 (worst min voltage). X-axis shows cumulative charge removed; Y-axis shows voltage.

Section 9 — Match-to-Match Performance Trends

Tracking key metrics across qualification matches in chronological order reveals whether the robot (or team) was improving during the event. Metrics: battery voltage trend (proxy for current draw and battery quality), CAN bus utilization (proxy for device count and communication health), average loop cycle time (proxy for software load), and peak CPU temperature.

Figure — Four key metrics across qualification matches (Q6 through Q82). Downward voltage trends indicate worse batteries or higher loads in later matches.

CAN Bus Health — All Matches

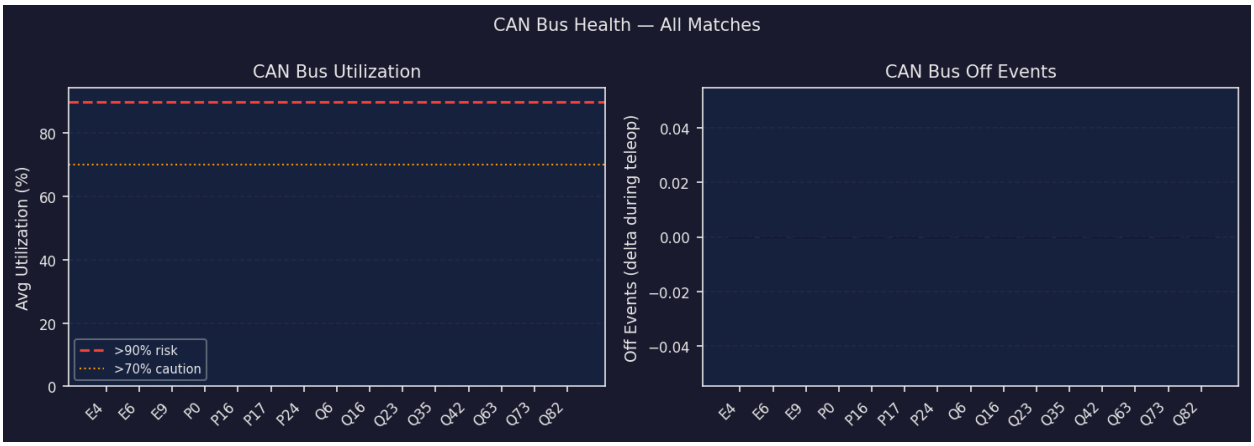

Figure — CAN bus utilization (left) and off-event count (right). Utilization above 90% risks dropped packets and missed actuator commands. Off events indicate a device briefly fell off the bus (connector issue or brownout).